

1. ALGORITHM & TEST-CASE TABLE

Algorithm Outline

- Main Loop:** Display menu options 1 to 10. Take choice input. Validate choice. Repeat until user selects 10.
- Geometrical Operations:**
 - Select solid type (1: Sphere, 2: Cylinder, 3: Cone, 4: Cuboid).
 - Validate choice. If Base Area is selected for Sphere, display error message and return to menu.
 - Take required inputs (radius, height, length, breadth) and call corresponding function from geometry.c.
- Physics Operations:**
 - Call relevant function from physics.c based on selected operation.
 - Calculate Mass, Weight, Potential Energy, Base Pressure, Buoyant Force, or Buoyancy Behavior.
- Math Functions:** Use math.h library (sqrt function) for slant height calculations.

Test-Case Table

Test ID	Operation / Module	Input Values	Expected Output / Result	Case Type
TC01	Geometry: Sphere Area	Radius = 3.0	113.097600 m ²	Valid
TC02	Geometry: Cone Area	Radius = 3.0, Height = 4.0	75.398400 m ² (Slant height = 5.0)	Valid
TC03	Geometry: Base Area	Sphere (Choice 1)	Error: Base area not defined for sphere	Invalid Selection
TC04	Physics: Buoyancy Behavior	Solid Density = 1000, Fluid = 1000	SUSPENDED (0)	Boundary
TC05	Physics: Buoyancy Behavior	Solid Density = 800, Fluid = 1000	FLOATS (-1)	Valid
TC06	Physics: Buoyancy Behavior	Solid Density = 2700, Fluid = 1000	SINKS (1)	Valid
TC07	Input Validation	Radius = -2.0	Error message, ask for input again	Invalid

2. COMPLETE C SOURCE CODE

File 1: solid.h

```
#ifndef SOLID_H
#define SOLID_H

#define PI 3.1416
#define GRAVITY 9.8

#define SPHERE 1
#define CYLINDER 2
#define CONE 3
#define CUBOID 4
```

```

#define FLOATS -1
#define SUSPENDED 0
#define SINKS 1

/* Declarations from geometry.c */
double sphere_area(double radius);
double sphere_volume(double radius);
double cylinder_area(double radius, double height);
double cylinder_volume(double radius, double height);
double cone_area(double radius, double height);
double cone_volume(double radius, double height);
double cuboid_area(double length, double breadth, double height);
double cuboid_volume(double length, double breadth, double height);
double circular_base_area(double radius);
double cuboid_base_area(double length, double breadth);

/* Declarations from physics.c */
double calculate_mass(double density);
double calculate_weight(void);
double potential_energy(double elevation);
double calculate_pressure(double density);
double buoyant_force(double fluid_density);
int determine_buoyancy(double solid_density, double fluid_density);

#endif

```

File 2: geometry.c

```

#include <stdio.h>
#include <math.h>
#include "solid.h"

double sphere_area(double radius) {
    return 4.0 * PI * radius * radius;
}

double sphere_volume(double radius) {
    return (4.0 / 3.0) * PI * radius * radius * radius;
}

double cylinder_area(double radius, double height) {
    return 2.0 * PI * radius * (radius + height);
}

double cylinder_volume(double radius, double height) {
    return PI * radius * radius * height;
}

double cone_area(double radius, double height) {
    double slant_height;
    slant_height = sqrt(radius * radius + height * height);
    return PI * radius * (radius + slant_height);
}

double cone_volume(double radius, double height) {
    return (1.0 / 3.0) * PI * radius * radius * height;
}

double cuboid_area(double length, double breadth, double height) {
    return 2.0 * (length * breadth + breadth * height + length * height);
}

```

```

double cuboid_volume(double length, double breadth, double height) {
    return length * breadth * height;
}

double circular_base_area(double radius) {
    return PI * radius * radius;
}

double cuboid_base_area(double length, double breadth) {
    return length * breadth;
}

```

File 3: physics.c

```

#include <stdio.h>
#include "solid.h"

static double get_positive_input(const char* prompt) {
    double val;
    do {
        printf("%s", prompt);
        scanf("%lf", &val);
        if (val <= 0) {
            printf("Error: Value must be positive. Try again.\n");
        }
    } while (val <= 0);
    return val;
}

static int get_solid_choice(void) {
    int choice;
    do {
        printf("Select Solid (1-Sphere, 2-Cylinder, 3-Cone, 4-Cuboid): ");
        scanf("%d", &choice);
        if (choice < 1 || choice > 4) {
            printf("Error: Invalid choice. Select between 1 and 4.\n");
        }
    } while (choice < 1 || choice > 4);
    return choice;
}

double calculate_mass(double density) {
    int choice = get_solid_choice();
    double vol = 0.0;
    double r, h, l, b;

    if (choice == SPHERE) {
        r = get_positive_input("Enter radius (m): ");
        vol = sphere_volume(r);
    } else if (choice == CYLINDER) {
        r = get_positive_input("Enter radius (m): ");
        h = get_positive_input("Enter height (m): ");
        vol = cylinder_volume(r, h);
    } else if (choice == CONE) {
        r = get_positive_input("Enter radius (m): ");
        h = get_positive_input("Enter height (m): ");
        vol = cone_volume(r, h);
    } else if (choice == CUBOID) {
        l = get_positive_input("Enter length (m): ");
        b = get_positive_input("Enter breadth (m): ");
        h = get_positive_input("Enter height (m): ");
    }
}

```

```

        vol = cuboid_volume(l, b, h);
    }

    return density * vol;
}

double calculate_weight(void) {
    double d = get_positive_input("Enter density (kg/m^3): ");
    double m = calculate_mass(d);
    return m * GRAVITY;
}

double potential_energy(double elevation) {
    double d = get_positive_input("Enter density (kg/m^3): ");
    double m = calculate_mass(d);
    return m * GRAVITY * elevation;
}

double calculate_pressure(double density) {
    int choice;
    double area = 0.0, vol = 0.0;
    double r, h, l, b;

    do {
        printf("Select Solid for Pressure (2-Cylinder, 3-Cone, 4-Cuboid): ");
        scanf("%d", &choice);
        if (choice == SPHERE) {
            printf("Error: Base area not defined for Sphere.\n");
        } else if (choice < 2 || choice > 4) {
            printf("Error: Invalid choice. Select 2, 3, or 4.\n");
        }
    } while (choice < 2 || choice > 4);

    if (choice == CYLINDER) {
        r = get_positive_input("Enter radius (m): ");
        h = get_positive_input("Enter height (m): ");
        area = circular_base_area(r);
        vol = cylinder_volume(r, h);
    } else if (choice == CONE) {
        r = get_positive_input("Enter radius (m): ");
        h = get_positive_input("Enter height (m): ");
        area = circular_base_area(r);
        vol = cone_volume(r, h);
    } else if (choice == CUBOID) {
        l = get_positive_input("Enter length (m): ");
        b = get_positive_input("Enter breadth (m): ");
        h = get_positive_input("Enter height (m): ");
        area = cuboid_base_area(l, b);
        vol = cuboid_volume(l, b, h);
    }

    return (density * vol * GRAVITY) / area;
}

double buoyant_force(double fluid_density) {
    int choice = get_solid_choice();
    double vol = 0.0;
    double r, h, l, b;

    if (choice == SPHERE) {
        r = get_positive_input("Enter radius (m): ");
        vol = sphere_volume(r);
    } else if (choice == CYLINDER) {

```

```

        r = get_positive_input("Enter radius (m): ");
        h = get_positive_input("Enter height (m): ");
        vol = cylinder_volume(r, h);
    } else if (choice == CONE) {
        r = get_positive_input("Enter radius (m): ");
        h = get_positive_input("Enter height (m): ");
        vol = cone_volume(r, h);
    } else if (choice == CUBOID) {
        l = get_positive_input("Enter length (m): ");
        b = get_positive_input("Enter breadth (m): ");
        h = get_positive_input("Enter height (m): ");
        vol = cuboid_volume(l, b, h);
    }

    return fluid_density * vol * GRAVITY;
}

int determine_buoyancy(double solid_density, double fluid_density) {
    if (solid_density > fluid_density) {
        return SINKS;
    } else if (solid_density < fluid_density) {
        return FLOATS;
    } else {
        return SUSPENDED;
    }
}
}

```

File 4: main.c

```

#include <stdio.h>
#include "solid.h"

static double get_pos(const char* prompt) {
    double val;
    do {
        printf("%s", prompt);
        scanf("%lf", &val);
        if (val <= 0) {
            printf("Error: Value must be positive. Try again.\n");
        }
    } while (val <= 0);
    return val;
}

static double get_non_neg(const char* prompt) {
    double val;
    do {
        printf("%s", prompt);
        scanf("%lf", &val);
        if (val < 0) {
            printf("Error: Elevation cannot be negative. Try again.\n");
        }
    } while (val < 0);
    return val;
}

int main(void) {
    int choice;
    int solid;
    double r, h, l, b, d, fd, elev, res;
    int b_res;
}

```

```

do {
    printf("\n===== MAIN MENU =====\n");
    printf("1. Surface Area\n");
    printf("2. Volume\n");
    printf("3. Base Area\n");
    printf("4. Mass\n");
    printf("5. Weight\n");
    printf("6. Gravitational Potential Energy\n");
    printf("7. Pressure on Base\n");
    printf("8. Buoyant Force\n");
    printf("9. Buoyancy Behavior\n");
    printf("10. Exit\n");
    printf("=====\n");
    printf("Enter choice (1-10): ");
    scanf("%d", &choice);

    if (choice < 1 || choice > 10) {
        printf("Error: Choice out of range. Enter 1 to 10.\n");
        continue;
    }

    switch (choice) {
        case 1:
            printf("\n-- Surface Area --\n");
            printf("Select Solid (1-Sphere, 2-Cylinder, 3-Cone, 4-Cuboid): ");
            scanf("%d", &solid);
            if (solid == SPHERE) {
                r = get_pos("Enter radius (m): ");
                res = sphere_area(r);
            } else if (solid == CYLINDER) {
                r = get_pos("Enter radius (m): ");
                h = get_pos("Enter height (m): ");
                res = cylinder_area(r, h);
            } else if (solid == CONE) {
                r = get_pos("Enter radius (m): ");
                h = get_pos("Enter height (m): ");
                res = cone_area(r, h);
            } else if (solid == CUBOID) {
                l = get_pos("Enter length (m): ");
                b = get_pos("Enter breadth (m): ");
                h = get_pos("Enter height (m): ");
                res = cuboid_area(l, b, h);
            } else {
                printf("Error: Invalid solid selection.\n");
                break;
            }
            printf("Surface Area = %.6f sq m\n", res);
            break;

        case 2:
            printf("\n-- Volume --\n");
            printf("Select Solid (1-Sphere, 2-Cylinder, 3-Cone, 4-Cuboid): ");
            scanf("%d", &solid);
            if (solid == SPHERE) {
                r = get_pos("Enter radius (m): ");
                res = sphere_volume(r);
            } else if (solid == CYLINDER) {
                r = get_pos("Enter radius (m): ");
                h = get_pos("Enter height (m): ");
                res = cylinder_volume(r, h);
            } else if (solid == CONE) {
                r = get_pos("Enter radius (m): ");
                h = get_pos("Enter height (m): ");
            }
    }
}

```

```

        res = cone_volume(r, h);
    } else if (solid == CUBOID) {
        l = get_pos("Enter length (m): ");
        b = get_pos("Enter breadth (m): ");
        h = get_pos("Enter height (m): ");
        res = cuboid_volume(l, b, h);
    } else {
        printf("Error: Invalid solid selection.\n");
        break;
    }
    printf("Volume = %.6f cu m\n", res);
    break;

case 3:
    printf("\n-- Base Area --\n");
    printf("Select Solid (1-Sphere, 2-Cylinder, 3-Cone, 4-Cuboid): ");
    scanf("%d", &solid);
    if (solid == SPHERE) {
        printf("Error: Base area for sphere is not defined.\n");
        break;
    } else if (solid == CYLINDER || solid == CONE) {
        r = get_pos("Enter radius (m): ");
        res = circular_base_area(r);
    } else if (solid == CUBOID) {
        l = get_pos("Enter length (m): ");
        b = get_pos("Enter breadth (m): ");
        res = cuboid_base_area(l, b);
    } else {
        printf("Error: Invalid solid selection.\n");
        break;
    }
    printf("Base Area = %.6f sq m\n", res);
    break;

case 4:
    printf("\n-- Mass --\n");
    d = get_pos("Enter solid density (kg/m^3): ");
    res = calculate_mass(d);
    printf("Mass = %.6f kg\n", res);
    break;

case 5:
    printf("\n-- Weight --\n");
    res = calculate_weight();
    printf("Weight = %.6f N\n", res);
    break;

case 6:
    printf("\n-- Potential Energy --\n");
    elev = get_non_neg("Enter elevation (m): ");
    res = potential_energy(elev);
    printf("Potential Energy = %.6f J\n", res);
    break;

case 7:
    printf("\n-- Pressure --\n");
    d = get_pos("Enter solid density (kg/m^3): ");
    res = calculate_pressure(d);
    printf("Pressure = %.6f Pa\n", res);
    break;

case 8:
    printf("\n-- Buoyant Force --\n");

```

```

        fd = get_pos("Enter fluid density (kg/m^3): ");
        res = buoyant_force(fd);
        printf("Buoyant Force = %.6f N\n", res);
        break;

    case 9:
        printf("\n-- Buoyancy Behavior --\n");
        d = get_pos("Enter solid density (kg/m^3): ");
        fd = get_pos("Enter fluid density (kg/m^3): ");
        b_res = determine_buoyancy(d, fd);
        if (b_res == SINKS) {
            printf("Result: Solid SINKS in the fluid.\n");
        } else if (b_res == FLOATS) {
            printf("Result: Solid FLOATS on the fluid.\n");
        } else {
            printf("Result: Solid remains SUSPENDED in the fluid.\n");
        }
        break;

    case 10:
        printf("Exiting program.\n");
        break;
}

} while (choice != 10);

return 0;
}

```